PRÁCTICA 10

Aplicación de tareas con Room

Memoria y análisis técnico de la aplicación



Mario Rapado Sánchez

PMDM · Desarrollo de Aplicaciones Multiplataforma

Octubre de 2026

Repositorio del proyecto

github.com/MarioRapSan/Practica10_TodoRoom

Aplicación original: github.com/AyatKhraisat/RoomExample

Contenido

1. Introducción

2. La aplicación

3. Estructura general de la aplicación

4. Componentes, almacenamiento y flujo

5. Cambio 1: pantalla «Acerca de»

6. Cambio 2: nueva paleta de colores

7. Cambio 3: sistema de prioridades

8. Pruebas realizadas

9. Cómo abrir y ejecutar el proyecto

10. Conclusiones

11. Enlaces y autoría

1. Introducción

Esta práctica parte de una aplicación Android ya existente, RoomExample, publicada por Ayat Khraisat. La aplicación sirve para crear una lista de tareas y guardarla en el propio teléfono.

El objetivo no era rehacer la aplicación, sino modificarla manteniendo su funcionamiento. Se han realizado tres cambios concretos:

- Añadir una pantalla «Acerca de».
- Cambiar la paleta de colores de los elementos visibles.
- Añadir un sistema de prioridades a las tareas.

El resto de la aplicación se conserva: crear tareas, verlas en una lista y eliminarlas. También se conservan los datos que ya existieran. Al abrir esta versión, las tareas antiguas siguen disponibles y pasan a tener prioridad media.

Resumen de los cambios

Cambio	Qué se ha hecho
1. Acerca de	Nueva pantalla con información de la aplicación, autoría, resumen de los cambios y enlace al repositorio original.
2. Paleta de colores	Sustituir los colores morado y rosa por una paleta de verde azulado, ámbar y verde claro.
3. Prioridades	Cada tarea puede ser alta, media o baja. La lista se ordena por prioridad y cada nivel tiene un color.

2. La aplicación

RoomExample es una lista de tareas sencilla. Al abrirla se ve una lista; si hay tareas guardadas, aparecen en tarjetas. El botón redondo de la esquina inferior derecha abre un cuadro para escribir una tarea nueva. Cada tarjeta tiene un botón para eliminarla.

Los datos no se guardan en un archivo de texto ni en Internet: se guardan en una base de datos interna del teléfono mediante Room. Esto significa que, al cerrar y volver a abrir la aplicación, las tareas siguen ahí.

Cómo se usa

- La pantalla principal pide a la base de datos la lista de tareas y la muestra.
- Al guardar una tarea, se escribe en la base de datos y la lista se vuelve a cargar.
- Al eliminar una tarea, se borra de la base de datos y la lista se actualiza.
- La pantalla «Acerca de» no modifica datos: solo muestra información.

Pantallas

Pantalla	Para qué sirve
Lista de tareas	Muestra las tareas guardadas y permite añadir o eliminar.
Añadir tarea	Cuadro de diálogo para escribir el texto y elegir la prioridad.
Acerca de	Información de la aplicación y enlace al proyecto original.

3. Estructura general de la aplicación

La aplicación se organiza como un proyecto Android habitual. El código Java está en app/src/main/java, los diseños y recursos en app/src/main/res, y la configuración de compilación en los archivos Gradle. Dentro del código hay tres paquetes: la raíz (pantallas, diálogo y adaptador), model (datos) y room (base de datos).

3.1 Árbol de archivos, carpetas y paquetes

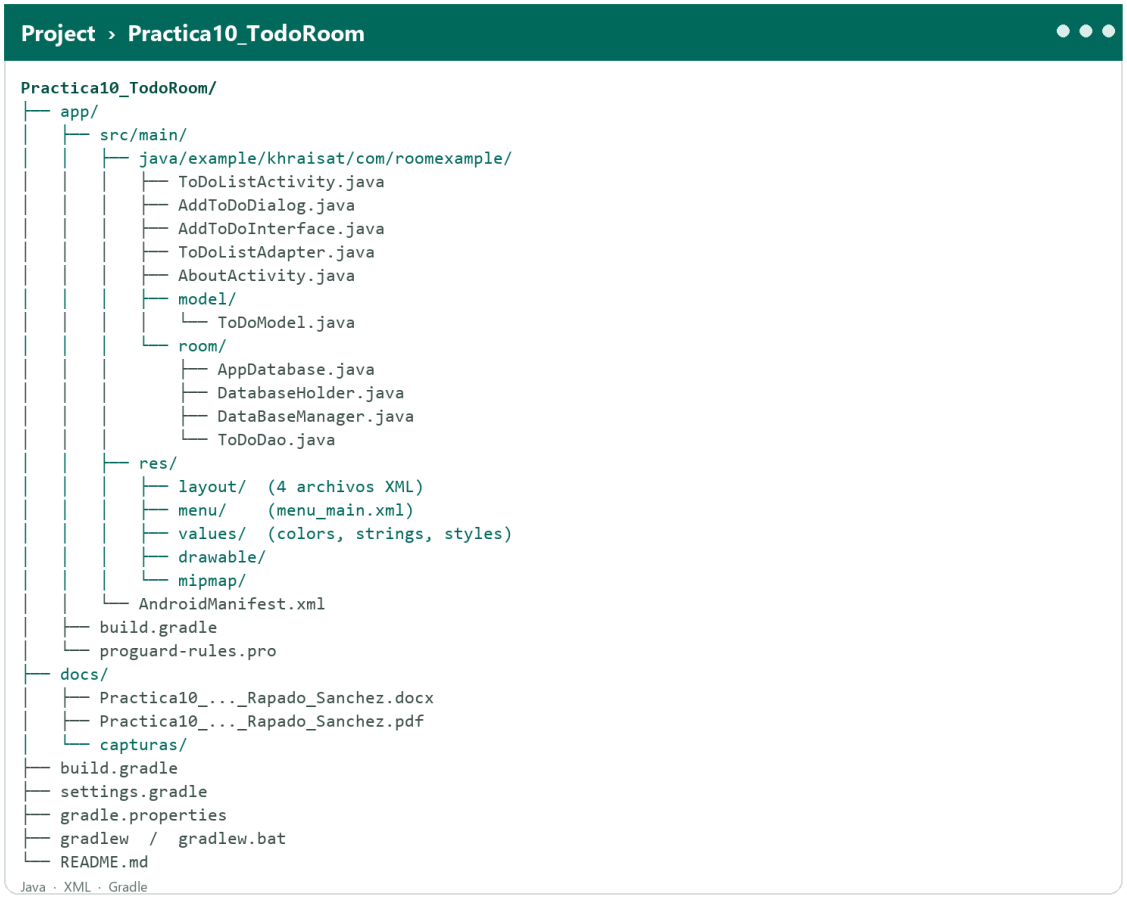


Figura 1. Árbol de archivos, carpetas y paquetes del proyecto.

En el árbol se distinguen las carpetas de código, recursos, documentación y configuración. El paquete raíz agrupa las dos pantallas, el diálogo, la interfaz de comunicación y el adaptador. El paquete model contiene ToDoModel, la entidad que representa una tarea. El paquete room reúne AppDatabase, ToDoDao, DatabaseHolder y DataBaseManager. Los recursos de res/layout, res/values y res/menu definen la interfaz, y los archivos Gradle configuran la compilación y las dependencias.

3.2 Clases principales y relación entre ellas

Las clases están separadas por responsabilidad. La pantalla principal coordina la interfaz, el diálogo y el adaptador; el adaptador prepara las tarjetas; y la capa room se encarga de guardar y recuperar los datos. El diagrama muestra las relaciones más importantes.

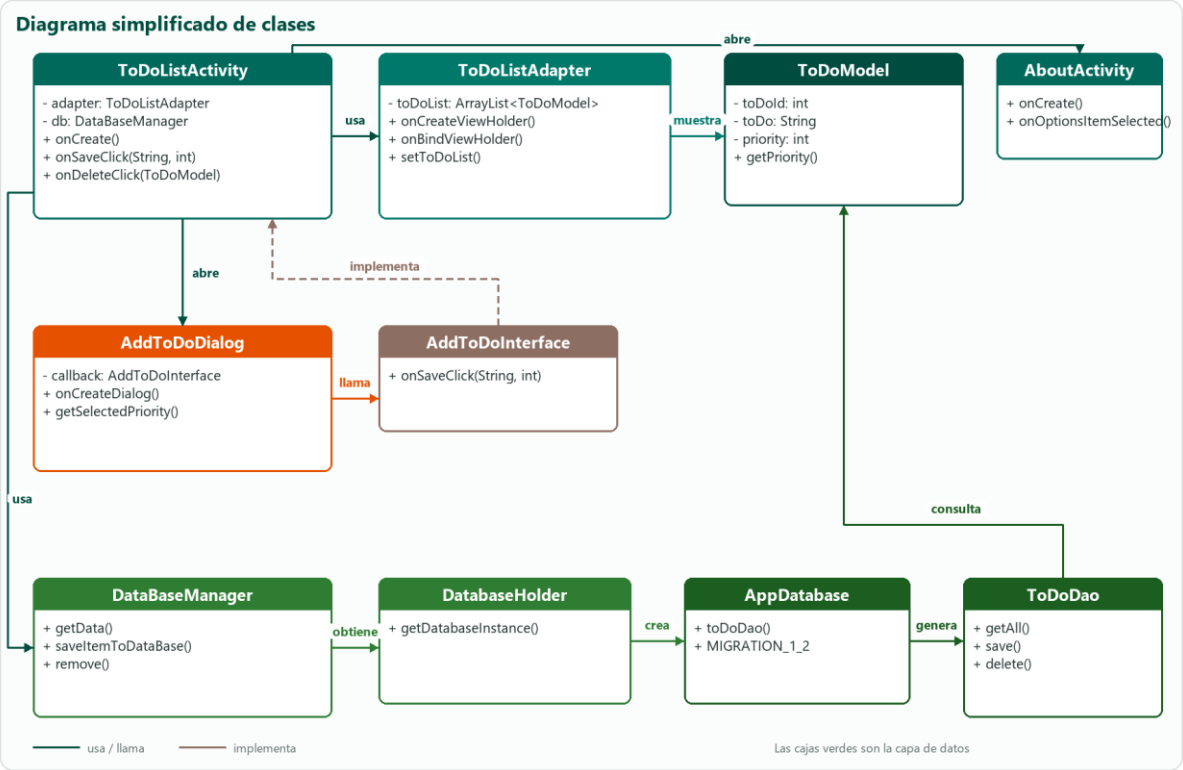


Figura 2. Diagrama simplificado de clases y relaciones principales.

Clase	Tipo	Función principal
ToDoListActivity	Activity	Carga la lista, abre el diálogo y «Acerca de», y atiende las acciones de guardar y eliminar.
AddToDoDialog + AddToDoInterface	DialogFragment + interfaz	Formulario de alta; devuelve el texto y la prioridad a la pantalla principal.
ToDoListAdapter	RecyclerView.Adapter	Crea cada tarjeta, muestra la prioridad y avisa cuando se pulsa DELETE.
AboutActivity	Activity	Muestra la información de la aplicación y permite abrir el repositorio o volver.
ToDoModel	Entidad Room	Guarda el identificador, el texto y la prioridad de una tarea.
ToDoDao	DAO	Define las consultas para leer, insertar y borrar tareas.
AppDatabase + DatabaseHolder	RoomDatabase + singleton	Definen la base de datos, la migración y una única instancia compartida.
DataBaseManager	Gestor	Agrupa las operaciones de la base de datos y las ejecuta en segundo plano.

4. Componentes, almacenamiento y flujo

4.1 Componentes Android utilizados

La aplicación utiliza los componentes básicos de Android. No hay servicios en segundo plano ni receptores de avisos: la interacción principal ocurre en las pantallas y en el diálogo.

Componente	Dónde se usa	Para qué sirve
Activity / AppCompatActivity	ToDoListActivity y AboutActivity	Representan las dos pantallas y gestionan su ciclo de vida.
DialogFragment	AddToDoDialog	Muestra el formulario de nueva tarea como un diálogo.
RecyclerView + Adapter	activity_to_do_list y ToDoListAdapter	Muestran la lista y reutilizan las tarjetas.
Layouts XML	res/layout (4 archivos)	Definen la estructura visual de pantallas, diálogo y tarjetas.
Menú de opciones	menu_main.xml	Añade la opción «Acerca de» a la barra superior.
Recursos	drawable, mipmap y values	Guardan iconos, colores, textos y el tema.
Room	Paquete room	Convierte objetos Java en filas de una base de datos SQLite.
Gradle y AndroidX	build.gradle	Configuran la compilación y las librerías de compatibilidad.

4.2 Almacenamiento y persistencia

La aplicación utiliza Room, la librería de persistencia de Android que trabaja sobre SQLite. No usa SharedPreferences, Firebase, archivos de texto ni una API REST. Los datos del usuario son las tareas; la configuración visual forma parte de los recursos de la aplicación.

Aspecto	Detalle
Sistema	Room sobre SQLite.
Base de datos	todo-database.
Ubicación	Almacenamiento privado de la app: /data/data/example.khraisat.com.roomexample/databases/todo-database.
Datos guardados	Una fila por tarea: toDoId, toDo y priority.
Configuración	Colores, textos y tema en res/values; no son datos del usuario.
Versión y migración	Versión 2; la migración 1→2 añade priority con valor 1.
Acceso	ToDoDao y DataBaseManager; DatabaseHolder crea una sola instancia.

Al ser una base de datos privada, otras aplicaciones no pueden leerla directamente. La configuración visual no se guarda como una preferencia del usuario: está definida en los recursos y se aplica al abrir la aplicación.

4.3 Flujo básico de ejecución

El recorrido principal, desde que se abre la aplicación hasta que se modifica una tarea, es el siguiente:

1. El lanzador abre ToDoListActivity.
2. onCreate() crea DataBaseManager, carga el layout y prepara RecyclerView y el botón flotante.
3. loadToDos() pide las tareas a DataBaseManager.
4. ToDoDao ejecuta la consulta y Room devuelve las filas como objetos ToDoModel.
5. ToDoListAdapter crea las tarjetas y muestra el texto y el color de la prioridad.
6. Al pulsar +, se abre AddToDoDialog; al pulsar SAVE, se llama a onSaveClick().
7. La actividad crea un ToDoModel, lo guarda y vuelve a cargar la lista.
8. Al pulsar DELETE, se borra la tarea y se recarga la lista; el menú abre AboutActivity.

La lista se recarga después de guardar y de eliminar para que el orden por prioridad y el contenido coincidan siempre con la base de datos.

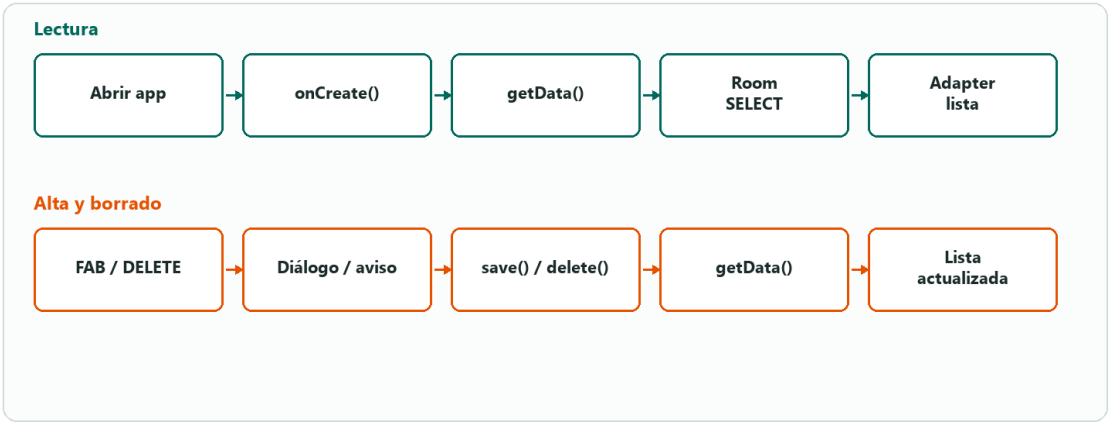


Figura 3. Flujo resumido de lectura, alta y borrado.

4.4 Componentes y librerías que quiero aprender

Algunos de estos componentes no los conocía antes de empezar. Estos son los que más me han llamado la atención y en los que me gustaría profundizar:

- Room: relaciones entre tablas, consultas más elaboradas y migraciones con varios cambios.
- RecyclerView y Adapter: ViewHolder, listas grandes y actualizaciones más precisas.
- Material Components y ConstraintLayout: construir pantallas adaptables y reutilizar estilos.
- AndroidX y AppCompat: entender cómo se mantiene la compatibilidad entre versiones de Android.
- Gradle y Android Gradle Plugin: organizar dependencias, versiones y tareas de compilación.
- AsyncTask: comprender mejor el trabajo en segundo plano y su relación con el ciclo de vida.

5. Cambio 1: pantalla «Acerca de»

Se ha añadido una pantalla nueva que se abre desde el menú de tres puntos de la barra superior. Su objetivo es que cualquier persona pueda ver de un vistazo qué aplicación está usando, de dónde viene y qué se ha cambiado.

Qué muestra

- El nombre de la aplicación y la versión.
- La autoría original y la persona que ha realizado la modificación.
- Un resumen de los tres cambios de esta práctica.
- Un botón para abrir el repositorio original en el navegador.
- Un botón «Volver» y una flecha de regreso a la lista.

La pantalla es solo informativa: no lee ni modifica las tareas. Se puede volver a la lista con la flecha de la barra superior o con el botón «Volver».

En el proyecto, la pantalla se llama AboutActivity y su diseño está en activity_about.xml. Se ha declarado en el manifiesto de Android y la opción del menú se define en menu_main.xml. El botón del repositorio abre el enlace con el navegador del teléfono; el botón «Volver» cierra la pantalla.



Figura 4. Pantalla «Acerca de» incluida en esta versión.

6. Cambio 2: nueva paleta de colores

Antes, la aplicación usaba una barra azul morada (#3F51B5), una barra de estado más oscura (#303F9F) y un botón flotante rosa (#FF4081). El fondo era blanco.

Ahora se usa una paleta basada en verde azulado, con un botón ámbar y un fondo verde muy claro. Los textos son oscuros para que se lean bien y el rojo se reserva para eliminar y para la prioridad alta.

Colores utilizados

Elemento	Antes	Ahora	Muestra
Barra superior	#3F51B5	**#00695C**	
Barra de estado y navegación	#303F9F	**#004D40**	
Botón flotante	#FF4081	**#FFB300**	
Botones y selección	#FF4081	**#E65100**	
Fondo	#FFFFFF	**#E8F5E9**	
Tarjetas	#FFFFFF	**#FFFFFF**	
Texto principal	#212121	**#1B2B2A**	
Botón eliminar	#FF4081	**#C62828**	

El cambio se aplica desde colors.xml y styles.xml. El tema usa el color principal en la barra superior, el color oscuro en la barra de estado y en la barra de navegación, y el fondo verde claro en las pantallas. Las tarjetas son blancas, con esquinas redondeadas y una sombra suave. El botón flotante es ámbar con el símbolo oscuro, y el botón DELETE es rojo para que se distinga del resto.

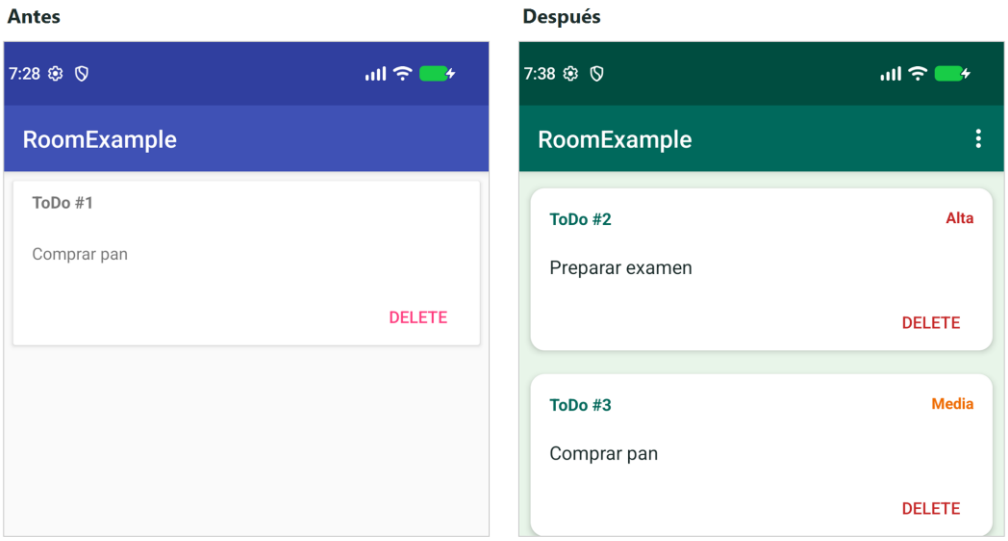


Figura 5. Comparación de la pantalla principal antes y después.

La nueva paleta también prepara la interfaz para el sistema de prioridades: el rojo, el naranja y el verde se distinguen bien sobre el fondo claro y sobre las tarjetas blancas.

7. Cambio 3: sistema de prioridades

Cada tarea tiene ahora un nivel: alta, media o baja. Al crear una tarea se elige en el mismo cuadro de diálogo. La opción marcada por defecto es media.

En la lista, cada tarjeta muestra su nivel en la esquina superior derecha y con un color propio. Además, la lista se ordena automáticamente: primero las tareas altas, después las medias y al final las bajas. Si hay varias del mismo nivel, se mantiene el orden de creación: primero la más antigua.

Niveles y colores

Prioridad	Se guarda como	Color	Muestra
Alta	2	#C62828	
Media	1	#EF6C00	
Baja	0	#2E7D32	

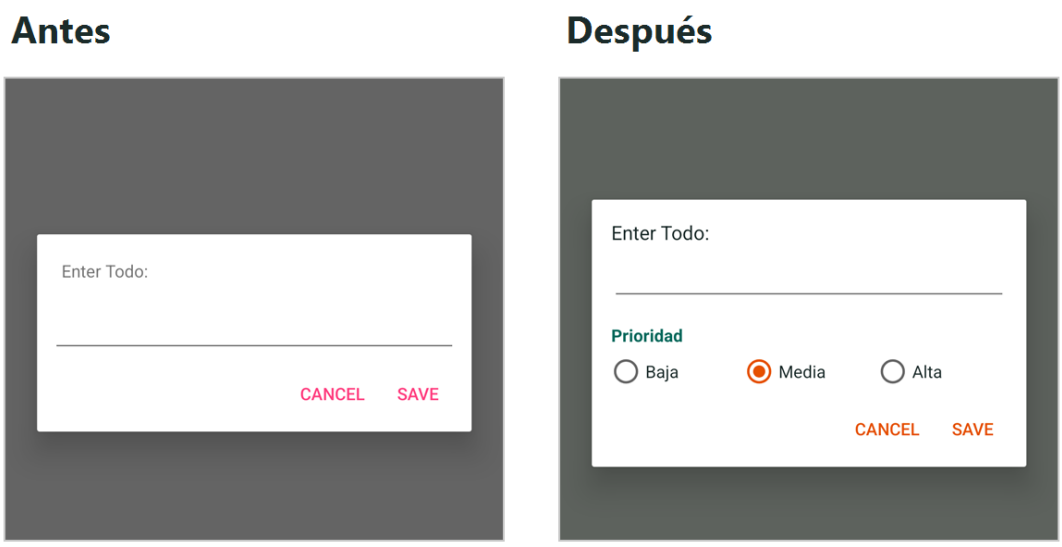


Figura 6. El diálogo original (izquierda) y el nuevo, con la elección de prioridad (derecha).

Cómo se usa

1. Pulsar el botón redondo con el signo +.
2. Escribir el texto de la tarea.
3. Elegir Baja, Media o Alta.
4. Pulsar SAVE. La tarea aparece en la posición que le corresponde.
5. Para eliminarla, pulsar DELETE. La lista se vuelve a ordenar.

En la lista, el resultado se ve así:



Figura 7. Lista ordenada por prioridad; el color y la etiqueta identifican cada nivel.

Cómo se guarda la prioridad

Se ha añadido un campo priority a cada tarea. Se guarda un número: 2 para alta, 1 para media y 0 para baja. Si una tarea no tiene valor, se toma 1 (media).

```
@ColumnInfo(defaultValue = "1")
private int priority = PRIORITY_MEDIUM;
```

La consulta pide las tareas ordenadas por ese número de mayor a menor. Así no hace falta ordenar la lista a mano cada vez que se abre la aplicación.

```
SELECT * FROM ToDoModel
ORDER BY priority DESC, toDoId ASC
```

La base de datos pasa de la versión 1 a la 2. Al abrir la aplicación con datos antiguos, se añade la columna nueva sin borrar nada; las tareas que ya existían quedan en media. Esto se probó con una tarea creada antes del cambio.

También se ajustó la lista para que se recargue al guardar y al eliminar. De esta forma el orden por prioridad se ve siempre y no se duplican tarjetas.

8. Pruebas realizadas

Las pruebas se hicieron en un emulador Pixel 9 con Android 17 (API 37). Se comprobó el funcionamiento normal, la conservación de los datos y los tres cambios.

Prueba	Resultado esperado	Resultado
Abrir la app con una tarea creada antes del cambio	La tarea sigue disponible y aparece como Media	**Correcto**
Crear una tarea con prioridad Alta	Aparece la primera y en rojo	**Correcto**
Crear una tarea con prioridad Baja	Aparece la última y en verde	**Correcto**
Cerrar y volver a abrir la aplicación	Se conservan las tareas y sus prioridades	**Correcto**
Eliminar una tarea	Desaparece y la lista se vuelve a ordenar	**Correcto**
Abrir Acerca de desde el menú	Se abre la pantalla y se puede volver	**Correcto**
Revisar los colores	Barra, fondo, tarjetas, botón y DELETE usan la nueva paleta	**Correcto**

Las capturas de las figuras 4 a 7 se tomaron del emulador y muestran el resultado real de la aplicación.

9. Cómo abrir y ejecutar el proyecto

Requisitos

- Android Studio actualizado.
- SDK de Android 36 instalado.
- Un emulador o un teléfono con Android 6.0 o superior.
- Conexión a Internet la primera vez, para descargar las dependencias.

Pasos

1. Descargar o clonar el repositorio.
2. Abrir la carpeta Practica10_TodoRoom con Android Studio.
3. Esperar a que termine la sincronización de Gradle.
4. Elegir el módulo app y un emulador o dispositivo.
5. Pulsar Run.

Repositorio: github.com/MarioRapSan/Practica10_TodoRoom

Aplicación original: github.com/AyatKhraisat/RoomExample

Nota: el proyecto original es de 2018. Para que abriera con Android Studio actual se actualizaron las herramientas de compilación (Gradle, AndroidX y Room). Es un ajuste necesario para compilar, no un cambio de funcionamiento ni una de las tres modificaciones de la práctica.

10. Conclusiones

Se han conseguido los tres objetivos de la práctica: la aplicación tiene una pantalla «Acerca de», usa una paleta de colores nueva y permite asignar prioridades a las tareas. Además, la lista se ordena según esa prioridad y los datos antiguos se conservan.

El funcionamiento básico no se ha roto: se pueden seguir creando tareas, viéndolas en la lista y eliminándolas. Las tareas que ya existían antes del cambio aparecen como media, sin tener que volver a escribirlas.

Esta práctica me ha servido para trabajar con un proyecto que no había escrito yo, entender cómo están separadas las pantallas, los datos y la base de datos, y hacer cambios sin rehacer la aplicación entera. También he comprobado la importancia de probar los cambios en un emulador y de cuidar los colores para que la información se lea bien.

11. Enlaces y autoría

Repositorios

Aplicación original: github.com/AyatKhraisat/RoomExample

Versión modificada: github.com/MarioRapSan/Practica10_TodoRoom

Documentación

Room en Android: developer.android.com/training/data-storage/room

Autoría

Aplicación original: Ayat Khraisat.

Modificación y memoria: Mario Rapado Sánchez.

Licencia del proyecto original: Apache 2.0.

Octubre de 2026.